GITAM UNIVERSITY

BENGALURU CAMPUS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ARTIFICIAL NEURAL NETWORKS (ANN)

PROJECT REPORT

Paddy Disease Representation using a Convolutional Autoencoder

Project No. 29 · Domain: Agriculture · Unit 2 / Unit 3 (CO2, CO3)

Submitted in partial fulfillment of the requirements for the course

ARTIFICIAL NEURAL NETWORKS

Submitted by

T Manoj

2023004133

Under the Guidance of

Anantha Babu

ACADEMIC YEAR 2026–2027

DECLARATION

I hereby declare that the project report entitled “Paddy Disease Representation using a Convolutional Autoencoder” is an original record of the work carried out by me as part of the Artificial Neural Networks course. The implementation, experiments and results reported in this document are based on the actual work performed for the project. All external sources and datasets have been properly acknowledged.

Student Name: T Manoj

USN / Roll No.: 2023004133

Signature: _____

Date: _____

CERTIFICATE

This is to certify that the project report entitled “Paddy Disease Representation using a Convolutional Autoencoder” has been prepared by T Manoj (2023004133) in partial fulfillment of the requirements of the Artificial Neural Networks course under my guidance.

Faculty Guide: Anantha Babu

Designation: _____

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I thank my faculty guide, Anantha Babu, for guidance throughout this project, and the Department of Computer Science and Engineering, GITAM University, Bengaluru Campus, for the opportunity to carry it out as part of the Artificial Neural Networks course.

I acknowledge Petchiammal A., Briskline Kiruba S., D. Murugan and Pandarasamy A., the creators of the Paddy Doctor dataset, and Kaggle for hosting the Paddy Disease Classification data. The project was built with the open-source PyTorch, scikit-learn, NumPy and Matplotlib libraries.

An AI coding assistant (Claude, Anthropic) was used to help write the code and draft this report. All experiments were run on the project code, and every reported number comes from those runs.

ABSTRACT

Diagnosing paddy diseases in the field is slow and depends on expert agronomists. This project investigates whether a convolutional autoencoder (CAE), trained only to reconstruct images, learns a compact representation that carries enough disease information to classify paddy images into ten classes (nine disease and pest-damage conditions plus normal). The 10,407 labelled images of the Kaggle Paddy Doctor dataset were checked for near-duplicates with a perceptual hash, split 70/15/15 with stratification and with duplicate clusters kept together, resized to 128 × 128 and scaled to [0, 1]. A three-layer convolutional encoder compresses each image into a 128-dimensional latent vector, and a mirrored decoder reconstructs it. The CAE was trained with plain mini-batch gradient descent on the mean-squared reconstruction error. The frozen encoder's latent vectors were then classified by a small dense network trained with class-weighted cross-entropy, also by mini-batch gradient descent. Experiments varied the learning rate, batch size and latent size, with momentum as a separate ablation. The selected configuration (learning rate 0.1, batch size 32, 150 epochs) reached a test reconstruction MSE of 0.0278 and a test macro-F1 of 0.475 (accuracy 0.484, macro AUC 0.847), compared with 0.370 for a PCA plus logistic-regression baseline of the same size. The learned representation therefore helps, but the reconstructions keep only colour and coarse layout rather than lesions, and the provisional 0.60 macro-F1 target was not reached.

Keywords: Artificial Neural Network, Agriculture, Paddy Doctor, Convolutional Autoencoder, Backpropagation, Mini-Batch Gradient Descent, Macro-F1

TABLE OF CONTENTS

1. Problem Statement 10

1.1 Background 10

1.2 Problem Definition 10

1.3 Objectives 10

1.4 ANN Concepts Used 11

1.5 Project Workflow 11

2. Dataset 12

2.1 Dataset Name and Source 12

2.2 Dataset Description 12

2.3 Dataset Summary 12

2.4 Feature Description 12

2.5 Dataset Visualization 13

3. Preprocessing 14

3.1 Data Cleaning 14

3.2 Encoding 14

3.3 Feature Scaling / Normalization 14

3.4 Outlier Handling 15

3.5 Train / Validation / Test Split 15

3.6 Preprocessing Summary 15

4. Neural Architecture 16

4.1 Architecture Overview 16

4.2 Layer Configuration 16

4.3 Neural Network Architecture Diagram 17

4.4 Forward Data Flow 17

4.5 Loss Function 17

4.6 Regularization 18
5. Optimization / Training 19
5.1 Selected Training Technique 19
5.2 Training Configuration 19
5.3 Backpropagation / Weight Update 19
5.4 Why This Training Method? 20
6. Experiments 21
6.1 Experimental Design 21
6.2 Experiment Configuration Table 21
6.3 Experiment Procedure 21
7. Results 22
7.1 Overall Performance 22
7.2 Training and Validation Accuracy 22
7.3 Training and Validation Loss 23
7.4 Confusion Matrix / Regression Metrics 24
7.5 Experiment Comparison 25
7.6 Reconstruction Quality and Latent Space 26
8. Discussion 28
8.1 Interpretation of Results 28
8.2 Effect of Architecture 28
8.3 Effect of Optimization / Training 28
8.4 Overfitting / Underfitting 29
8.5 Objective Achievement 29
9. Limitations 30
10. Conclusion 31
10.1 Future Work 31

11. References..... 32

APPENDIX A – SOURCE CODE / IMPORTANT CODE SNIPPETS..... 33

APPENDIX B – ADDITIONAL RESULTS..... 34

LIST OF FIGURES

Figure 1. Project workflow

Figure 2. Dataset class distribution

Figure 3. Convolutional autoencoder and classifier architecture

Figure 4. Classifier head training and validation accuracy

Figure 5. Training and validation loss (autoencoder and classifier head)

Figure 6. Confusion matrix on the test split

Figure 7. Test images and their reconstructions

Figure 8. t-SNE of test latent vectors

LIST OF TABLES

Table 1. Dataset summary

Table 2. Feature description

Table 3. Preprocessing steps

Table 4. ANN architecture

Table 5. Training configuration

Table 6. Experiment configurations

Table 7. Actual experimental results (test split)

Table 8. Per-class results on the test split

Table 9. Comparison of all experiments (validation split)

Table 10. Objective achievement

Table 11. Experiment log

1. Problem Statement

1.1 Background

Rice (paddy) is a staple crop, and diseases and pests such as blast, bacterial leaf blight, tungro and hispa reduce yield when they are not caught early. In the field, diagnosis usually means an expert inspecting the plants, which is slow and not always available to farmers. Image-based models could give a first diagnosis from a phone photograph.

This project looks at one part of that question: whether a neural network can learn, without labels, a compact numerical description of a paddy image that still separates the diseases. If it can, the same representation could later be reused for search, clustering or classification with few labels.

1.2 Problem Definition

- **Input:** an RGB field photograph of paddy plants (480 × 640 pixels), resized to 128 × 128 × 3.
- **Output / target:** one of 10 labels: bacterial_leaf_blight, bacterial_leaf_streak, bacterial_panicle_blight, blast, brown_spot, dead_heart, downy_mildew, hispa, normal, tungro.
- **Learning task:** (1) unsupervised representation learning: an encoder maps an image x to a latent vector z of 128 values and a decoder rebuilds x from z; (2) supervised multi-class classification from z, with the encoder frozen.
- **Constraints:** the assigned architecture (convolutional autoencoder, trained from scratch, no pretrained backbones or transfer learning) and training method (mini-batch gradient descent); only the labelled Kaggle training images can be scored, because the Kaggle test images are unlabelled; the classes are imbalanced (5.2 : 1); training on a single laptop GPU (NVIDIA RTX 3050).

1.3 Objectives

- To prepare the Paddy Doctor images for training: duplicate check, leakage-free split, resizing and scaling.
- To design and implement a convolutional autoencoder that reconstructs paddy images from a low-dimensional latent vector.

- To train it with mini-batch gradient descent and document batch size, learning rate and convergence.
- To classify the 10 classes from the frozen encoder's latent vectors and compare against a simple baseline of the same size.
- To report reconstruction loss, a latent-space visualisation, macro-F1 and per-class results with honest observations.

1.4 ANN Concepts Used

- **Convolutional layers:** local receptive fields and weight sharing, with stride 2 for down-sampling.
- **Autoencoder:** an encoder–bottleneck–decoder network trained to reproduce its input (unsupervised learning).
- **Transposed convolutions:** learned up-sampling in the decoder.
- **Multilayer perceptron (shallow ANN):** the dense classifier head on top of the latent vector.
- **Activation functions:** ReLU in hidden layers, sigmoid for the reconstructed pixels, softmax for class probabilities.
- **Backpropagation and mini-batch gradient descent:** gradient computation and the weight update.
- **He (Kaiming) normal initialisation** and **early stopping** on validation loss.
- **Momentum:** used only in a labelled side comparison (ablation).

1.5 Project Workflow

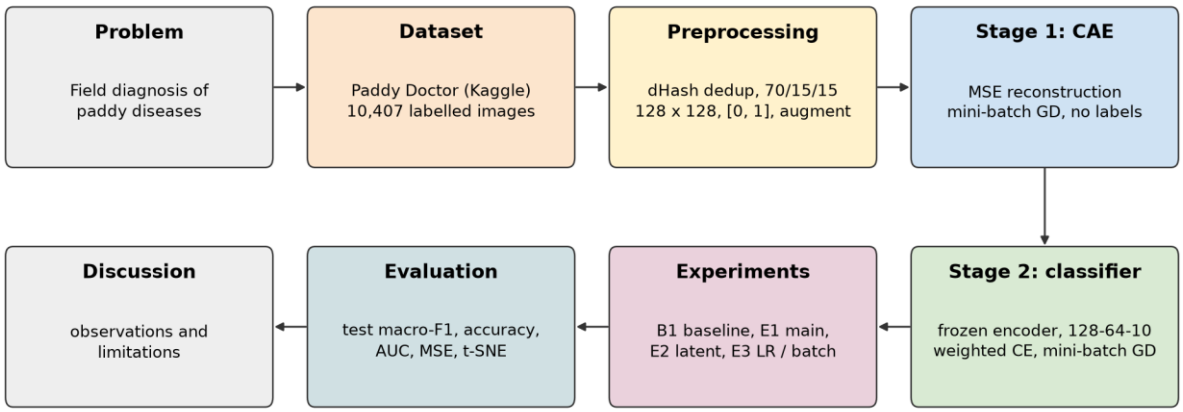


Figure 1. Project workflow

2. Dataset

2.1 Dataset Name and Source

Dataset: Paddy Doctor: Paddy Disease Classification (Kaggle competition data) [1], [2]

Source: https://www.kaggle.com/competitions/paddy-disease-classification

Domain: Agriculture (plant disease diagnosis)

2.2 Dataset Description

The labelled part of the data contains 10,407 RGB JPEG photographs, each 480 × 640 pixels (landscape or portrait). They are field shots of paddy plants taken with a smartphone, not close-ups of single leaves. The file train.csv lists, for every image, its label, the paddy variety (10 varieties) and the plant age (45 to 82 days). A further 3,470 Kaggle test images have no labels and were not used.

The target is the label column, with 10 classes: nine disease or pest-damage conditions and normal. Only the pixels are used as model input; variety and age are not.

2.3 Dataset Summary

Table 1. Dataset summary

Property	Value	Source/Method	Remarks
Number of samples	10,407 labelled images	train.csv and train_images/	3,470 unlabelled Kaggle test images not used
Number of features	49,152 (128 × 128 × 3 pixels)	After resizing (raw: 480 × 640 × 3)	Metadata (variety, age) excluded
Target variable	label (disease class)	train.csv	One label per image
Classes / output range	10 classes	EDA (01_eda_split.py)	Imbalanced: 1,764 (normal) to 337 (bacterial panicle blight), 5.2 : 1

2.4 Feature Description

Table 2. Feature description

Feature	Data type	Meaning	Role
Image pixels	uint8 RGB, 480 × 640 × 3	Field photograph of paddy plants	Model input
label	Categorical (10 values)	Disease class assigned by experts	Target
variety	Categorical (10 values)	Paddy variety	Not used (metadata)
age	Integer (45–82)	Plant age in days	Not used (metadata)
image_id	String	Image file name	Identifier only

2.5 Dataset Visualization

Figure 2 shows the number of images per class. The four largest classes (normal, blast, hispa, dead_heart) each have more than 1,400 images, while the three bacterial classes have 337 to 479. This imbalance is why macro-F1, which weights every class equally, is the headline metric.

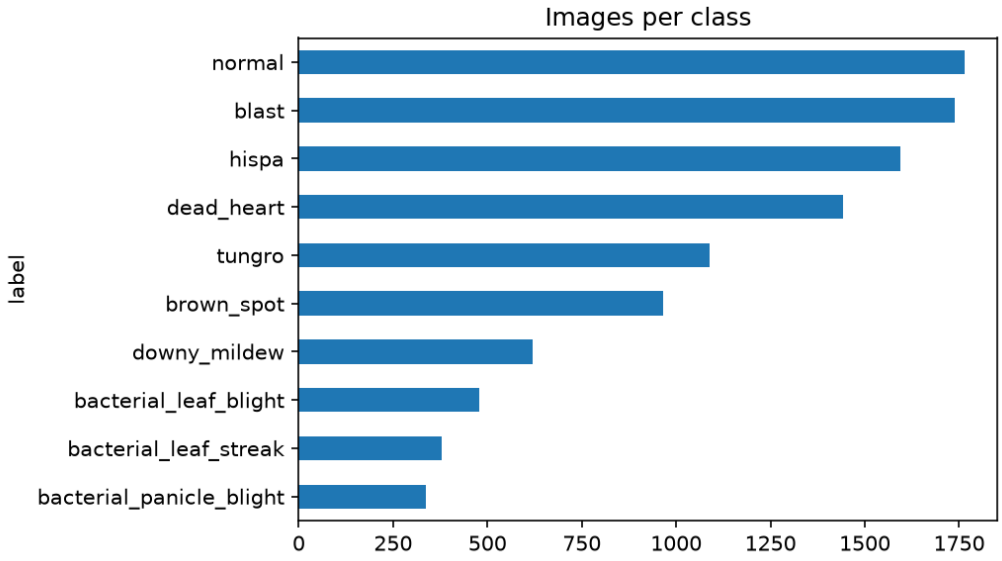


Figure 2. Dataset class distribution

3. Preprocessing

3.1 Data Cleaning

- **Missing values:** train.csv has no missing values (0 in all four columns), no duplicate image IDs, and all 10,407 images load correctly.
- **Duplicate records:** each image received a 64-bit difference hash (dHash), and images with a Hamming distance of 2 or less were linked into clusters. 1,203 images (11.6%) belong to a cluster of two or more, giving 9,697 clusters, and 10 clusters contain mixed labels. Instead of deleting them, each cluster is kept inside one split (Section 3.5), so a near-copy of a test image is never seen in training.
- **Irrelevant attributes:** variety and age are not used as input, and image_id is an identifier only.
- **Invalid values:** none found. Only two image sizes occur (480 × 640 and 640 × 480).

3.2 Encoding

The text labels were mapped to integers 0–9 in alphabetical order (bacterial_leaf_blight = 0, …, tungro = 9). The cross-entropy loss takes these integer targets directly, which is equivalent to one-hot encoding. There are no other categorical inputs.

3.3 Feature Scaling / Normalization

Each image was resized to 128 × 128 with bilinear interpolation (this does not keep the aspect ratio) and its pixel values were divided by 255, a min-max scaling to [0, 1]. This range matches the decoder's sigmoid output, so the reconstruction error compares like with like. No per-channel mean/std normalisation was applied to the images.

For Stage 2, each latent dimension was standardised with the mean μ and standard deviation σ of the training latents only, because plain gradient descent is sensitive to features on different scales:

$$x' = \frac{x}{255} \in [0, 1], \qquad \tilde{z}_j = \frac{z_j - \mu_j}{\sigma_j}$$

Training batches only were augmented with a random horizontal flip, a rotation within ±15° and a brightness change within ±20%. Validation and test images were never augmented.

3.4 Outlier Handling

Pixel values are bounded (0 to 255), so no outlier treatment was needed. The data-quality issues that matter for images, near-duplicates and mixed-label clusters, were handled as described in Section 3.1.

3.5 Train / Validation / Test Split

The split was made first, before anything was fitted, using StratifiedGroupKFold with 20 folds and random seed 42: 3 folds for test, 3 for validation and 14 for training. The groups are the duplicate clusters, so no cluster is split. The result is 7,282 training (70.0%), 1,561 validation (15.0%) and 1,564 test (15.0%) images, with the same class proportions in each.

Everything statistical (PCA for the baseline, the latent mean and standard deviation, the class weights) was fitted on the training split only. Model choices were made on the validation split, and the test split was used once, for the final evaluation. The split indices are saved to artifacts/split.npz and reused by every experiment.

3.6 Preprocessing Summary

Table 3. Preprocessing steps

Step	Method Used	Reason
Missing values	None present (0 nulls); no imputation needed	Verified in EDA
Duplicates	dHash, Hamming ≤ 2; each cluster kept in one split	Prevents an inflated test score
Categorical encoding	Label → integer 0–9	Required by the cross-entropy loss
Resizing	480 × 640 → 128 × 128, bilinear	Fits GPU memory and training time
Scaling	Pixels ÷ 255 → [0, 1]; latents z-scored with train statistics	Matches the sigmoid output; stable gradient descent
Augmentation	Flip, ±15° rotation, ±20% brightness (train only)	More variety without leakage
Data split	70 / 15 / 15, stratified, grouped by cluster, seed 42	Same class ratios; no leakage; reproducible

4. Neural Architecture

4.1 Architecture Overview

The assigned architecture is a convolutional autoencoder (CAE). Convolutions suit images because each filter looks at a small neighbourhood and is reused across the whole image. The three encoder convolutions have only 93,248 weights in total, whereas a single dense layer on the 49,152 input pixels would need millions. The autoencoder learns without labels: it must squeeze each image through a 128-value bottleneck and rebuild it, so the bottleneck has to keep the information that matters most for reconstruction.

The project has two stages. Stage 1 trains the encoder and decoder together on reconstruction. Stage 2 freezes the encoder, discards the decoder, and trains a small dense classifier head on the 128-value latent vectors. Comparing that classifier with a baseline of the same size shows whether the unsupervised representation carries disease information.

4.2 Layer Configuration

Table 4. ANN architecture

Layer	Neurons / Shape	Activation	Purpose	Parameters
Input	128 × 128 × 3	—	Receive the RGB image	0
Conv 1 (3×3, stride 2, 32 filters)	64 × 64 × 32	ReLU	Low-level edges and colour	896
Conv 2 (3×3, stride 2, 64 filters)	32 × 32 × 64	ReLU	Mid-level texture	18,496
Conv 3 (3×3, stride 2, 128 filters)	16 × 16 × 128	ReLU	Higher-level patterns	73,856
Flatten + Dense (bottleneck)	128 (latent z)	Linear	Compressed representation	4,194,432
Decoder Dense + reshape	16 × 16 × 128	ReLU	Expand the latent vector	4,227,072
ConvTranspose 1 (3×3, stride 2)	32 × 32 × 64	ReLU	Up-sample	73,792
ConvTranspose 2 (3×3, stride 2)	64 × 64 × 32	ReLU	Up-sample	18,464
ConvTranspose 3 (3×3, stride 2)	128 × 128 × 3	Sigmoid	Reconstructed image	867
Head Dense 1	64	ReLU	Classifier hidden layer	8,256
Head Dense 2 (output)	10	Softmax	Class probabilities	650
Total	**Encoder 4,287,680 · Decoder 4,320,195 · Head 8,906**			**8,616,781**

All convolutions use “same” padding, so each stride-2 layer halves the height and width. Almost all parameters sit in the two dense layers that connect the 32,768-value feature map to the 128-value latent vector and back.

4.3 Neural Network Architecture Diagram

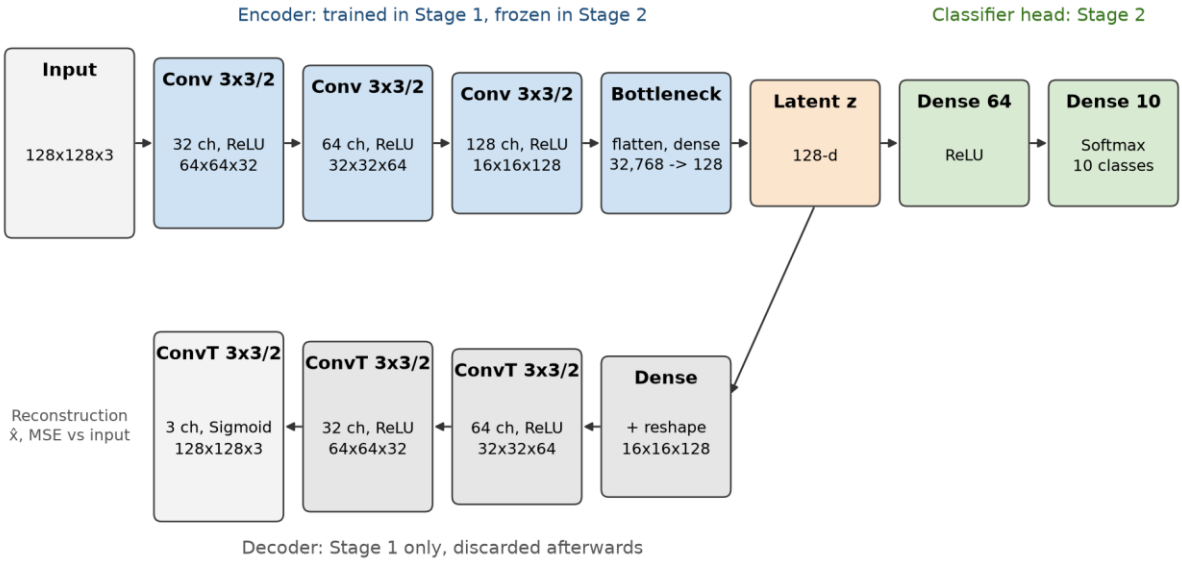


Figure 3. Convolutional autoencoder and classifier architecture

4.4 Forward Data Flow

An image enters as a 128 × 128 × 3 array of values in [0, 1]. Each encoder convolution applies its filters, adds a bias and passes the result through ReLU, halving the spatial size: 64 × 64 × 32, then 32 × 32 × 64, then 16 × 16 × 128. That map is flattened to 32,768 values and a dense layer projects it to the 128-value latent vector z.

In Stage 1, the decoder maps z back to 32,768 values, reshapes them to 16 × 16 × 128, and three transposed convolutions double the size each time back to 128 × 128 × 3. A sigmoid keeps the output pixels in [0, 1]. In Stage 2, z is standardised and passed through the head: a 64-neuron ReLU layer and a 10-neuron output layer whose softmax gives the class probabilities. The predicted class is the one with the highest probability.

4.5 Loss Function

Stage 1, mean squared error (MSE) between each image x and its reconstruction $\hat{x}$, averaged over the N images in a batch and all C × H × W pixel values:

$$L_{MSE} = \frac{1}{NCHW} \sum_{n=1}^{N} \sum_{c,h,w} \left(x_{n,c,h,w} - \hat{x}_{n,c,h,w}\right)^2$$

MSE is the standard reconstruction loss for real-valued pixels and needs no labels. It does tend to favour blurry, average-looking outputs, which matters for the results (Section 7.6).

Stage 2, class-weighted cross-entropy. Each class k gets a weight w_k inversely proportional to its training count N_k (K = 10 classes, N training images), so that mistakes on the small bacterial classes cost as much as mistakes on the large ones:

$$L_{CE} = -\frac{\sum_n w_{y_n} \log p_{n,y_n}}{\sum_n w_{y_n}}, \qquad p_n = \mathrm{softmax}(h(z_n)), \qquad w_k = \frac{N}{K N_k}$$

4.6 Regularization

No dropout, batch normalisation or L2 weight decay was used, so that the network matches the assigned plain architecture and optimiser. Overfitting is controlled instead by:

- **Early stopping** on validation loss (patience 8), with the best weights restored.
- **Data augmentation** of training batches (flip, rotation, brightness).
- **The bottleneck itself:** 128 values cannot memorise 49,152-value images.

5. Optimization / Training

5.1 Selected Training Technique

Both stages use **plain mini-batch gradient descent with backpropagation** (PyTorch torch.optim.SGD with momentum 0 and no weight decay). In each step the parameters θ move against the loss gradient averaged over a mini-batch B_t of m examples, with learning rate η:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{m} \sum_{i \in B_t} \nabla_\theta L(x_i;\, \theta_t)$$

The training set is reshuffled every epoch and split into consecutive mini-batches, so each epoch visits every training image once.

5.2 Training Configuration

Table 5. Training configuration

Parameter	Actual Value
Optimizer / training method	Mini-batch gradient descent (SGD, momentum 0), both stages
Learning rate	Stage 1: 0.1 · Stage 2: 0.1 (both chosen from 0.1 / 0.01 / 0.001)
Batch size	Stage 1: 32 (chosen from 32 / 64 / 128) · Stage 2: 64
Epochs	Stage 1: 150 (best epoch 150) · Stage 2: 19 run, best epoch 11 (maximum 50)
Loss function	Stage 1: MSE · Stage 2: class-weighted cross-entropy
Weight initialization	He (Kaiming) normal, biases zero
Early stopping	Patience 8 on validation loss, best weights restored (both stages)
Latent size	128 (chosen from 64 / 128 / 256)
Random seed	42 (split, initialisation, shuffling)
Hardware / software	NVIDIA RTX 3050 Laptop GPU; PyTorch 2.14 (CUDA 12.6), scikit-learn
Time per epoch	Stage 1: 12.2 s (GPU shared with a second run; about 5.4 s at batch 64 when alone) · Stage 2: under 1 s

5.3 Backpropagation / Weight Update

- **Forward pass:** a mini-batch flows through the network as in Section 4.4, and every layer keeps its inputs and pre-activations.
- **Loss calculation:** the MSE (Stage 1) or weighted cross-entropy (Stage 2) is computed for the batch.

- **Gradient computation:** backpropagation applies the chain rule from the output layer back to the first layer. For layer l with weights W, pre-activation s, activation a and activation function g, the error signal δ and the weight gradient are:

$$\delta^{(l)} = (W^{(l+1)T}\delta^{(l+1)}) \odot g'(s^{(l)}), \qquad \frac{\partial L}{\partial W^{(l)}} = \delta^{(l)}\, a^{(l-1)T}$$

- **Weight update:** each parameter takes one step against its batch-averaged gradient (Section 5.1). PyTorch's autograd computes these gradients automatically, including for convolutions, where the gradient of a shared filter is summed over every position it was applied to.
- **Stage 2:** the encoder is frozen, so gradients only reach the 8,906 head parameters.

5.4 Why This Training Method?

Mini-batch gradient descent was assigned, and it also fits this problem. Full-batch gradient descent would need all 7,282 training images (49,152 values each) and their activations in GPU memory at once, which a 4 GB laptop GPU cannot hold. Per-image updates (pure stochastic gradient descent) would waste the GPU's parallelism. Mini-batches of 32 to 128 images fit in memory, give 57 to 228 updates per epoch, and let each batch be augmented independently.

Using plain gradient descent without momentum or adaptive learning rates makes the effect of the learning rate and batch size directly visible, which is the point of experiment E3. Its weakness, slow convergence, was a known risk; momentum is reported only as a separate ablation.

6. Experiments

6.1 Experimental Design

Kept constant across experiments: the saved split, seed 42, the preprocessing, early stopping (patience 8), and the encoder depth. When encoders were compared, the head was always trained the same way (learning rate 0.01, batch size 64), so differences come from the encoder.

Changed one factor at a time, judged on the validation split: the learning rate, the batch size and the latent size of the autoencoder, each for 50 epochs. Because every plain-SGD run was still improving at 50 epochs, the selected configuration was retrained for 150 epochs, and then the head's learning rate was tuned. Momentum 0.9 is a labelled side comparison only. The optional fine-tuning experiment (E4, encoder unfrozen) was not run.

6.2 Experiment Configuration Table

Table 6. Experiment configurations

Exp.	Architecture	Learning Rate	Batch Size	Epochs	Purpose
B1	PCA (128) on 64 × 64 pixels + logistic regression	n/a (L-BFGS)	Full batch	≤ 2,000 iterations	Baseline
E1	CAE, z = 128, + head 128–64–10	CAE 0.1 · head 0.1	CAE 32 · head 64	CAE 150 · head 19	Main model
E2	CAE, z = 64 and z = 256	0.1	64	50	Test architecture (latent size)
E3a	CAE, z = 128	0.1 / 0.01 / 0.001	64	50	Test learning rate
E3b	CAE, z = 128	0.1	32 / 64 / 128	50	Test batch size
M	CAE, z = 128, momentum 0.9	0.1 / 0.01	64	50 (and 150 at 0.1)	Momentum ablation
H	Head on the E1 encoder	0.1 / 0.01 / 0.001	64	≤ 50 (early stop)	Head learning rate

6.3 Experiment Procedure

1. Prepare the fixed train/validation/test split.
2. Train the baseline model.
3. Record training and validation metrics.
4. Change one experimental factor at a time where practical.
5. Evaluate every final model on the same held-out test set.
6. Record actual results without fabricating or selectively omitting outcomes.

7. Results

7.1 Overall Performance

Table 7 gives the final results. Precision, recall and F1 are macro-averaged over the 10 classes, and AUC is the macro one-vs-rest ROC AUC. Only B1 and E1 were evaluated on the test split, once, so that the test set stayed untouched during tuning. The other experiments are compared on the validation split in Section 7.5.

Table 7. Actual experimental results (test split)

Experiment	Accuracy	Precision	Recall	F1	AUC	Notes
B1 (test)	0.389	0.365	0.401	0.370	0.792	PCA-128 + logistic regression baseline
E1 (test)	0.484	0.465	0.504	0.475	0.847	+0.105 macro-F1 over B1; below the 0.60 target
B1 (validation)	0.402	0.387	0.447	0.398	—	Reference
E1 (validation)	0.524	0.512	0.554	0.520	—	Optimistic: used for model selection

E1 beats the baseline on every metric, by 0.105 macro-F1 and 0.055 AUC, at the same representation size of 128 values. The PRD's acceptance criterion E1 > B1 is met; the provisional target of 0.60 macro-F1 is not. The test reconstruction MSE of the autoencoder is 0.0278.

7.2 Training and Validation Accuracy

The autoencoder has no accuracy, because Stage 1 has no labels. Figure 4 shows the classifier head's accuracy per epoch. Training accuracy (measured on the training batches) climbs steadily to 0.720, while validation accuracy rises more slowly, reaching 0.524 at the best validation-loss epoch (11) and peaking at 0.562 at epoch 18.

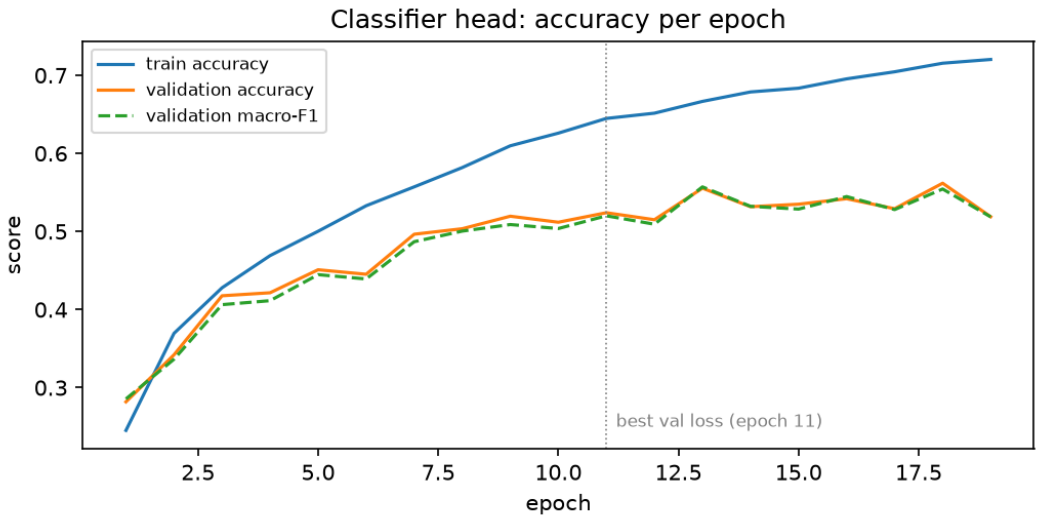


Figure 4. Classifier head training and validation accuracy

7.3 Training and Validation Loss

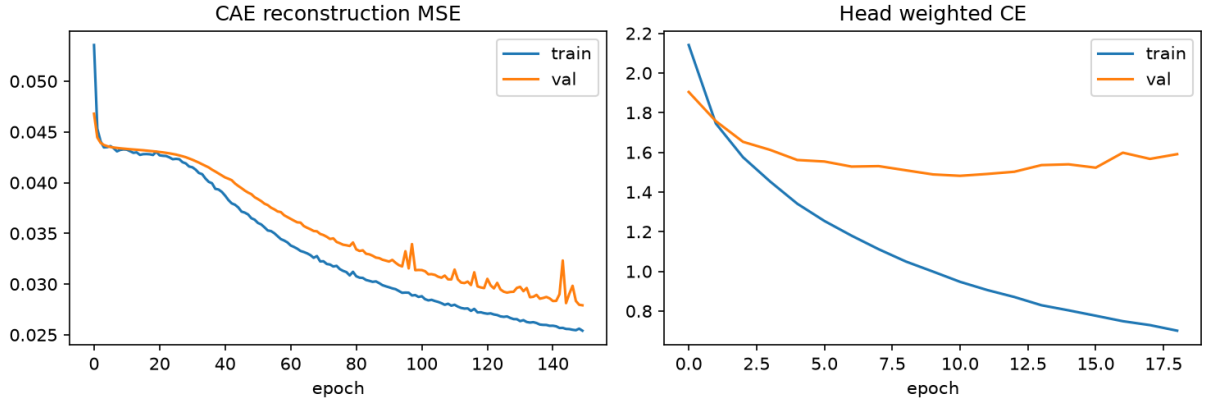


Figure 5. Training and validation loss (left: autoencoder MSE; right: classifier head)

- **Autoencoder plateau:** with plain gradient descent the MSE drops quickly to about 0.043 and then stays flat for about 25 epochs. That level matches a network that outputs roughly an average green image.
- **Break-out:** after about epoch 25 the loss falls steadily again. This plateau explains why the 50-epoch runs all looked similar, and why training longer mattered more than any single hyperparameter.
- **Generalisation:** final MSE is 0.0254 on training, 0.0279 on validation and 0.0278 on test, so the validation curve tracks the training curve closely. Small validation spikes after about epoch 90 suggest a learning rate of 0.1 is close to the stable limit.
- **Classifier head:** training loss keeps falling, but validation loss bottoms out at epoch 11 and then rises, so the head starts to overfit and early stopping restores the epoch-11 weights.

7.4 Confusion Matrix / Regression Metrics

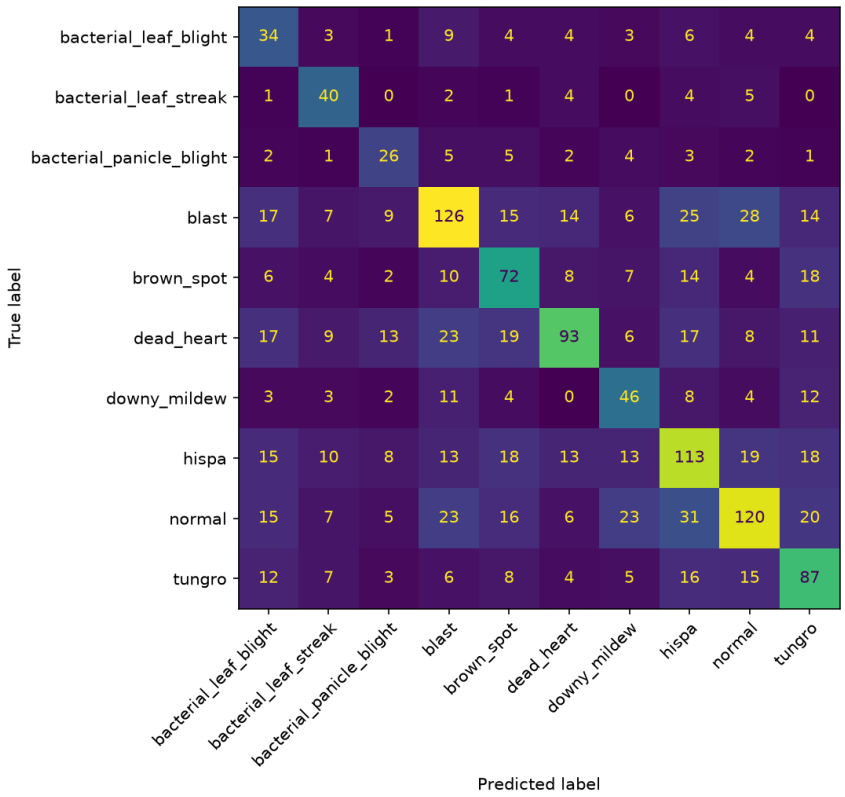


Figure 6. Confusion matrix on the test split

Table 8. Per-class results on the test split

Class	Precision	Recall	F1	Support
bacterial_leaf_blight	0.279	0.472	0.351	72
bacterial_leaf_streak	0.440	0.702	0.541	57
bacterial_panicle_blight	0.377	0.510	0.433	51
blast	0.553	0.483	0.515	261
brown_spot	0.444	0.497	0.469	145
dead_heart	0.628	0.431	0.511	216
downy_mildew	0.407	0.495	0.447	93
hispa	0.477	0.471	0.474	240
normal	0.574	0.451	0.505	266
tungro	0.470	0.534	0.500	163
Macro average	**0.465**	**0.504**	**0.475**	**1,564**

- **bacterial_leaf_blight (F1 0.351)** is the weakest class. Recall is acceptable (0.472), but precision is only 0.279: many blast, dead_heart, hispa and normal images are predicted as this class. The class weights push predictions toward small classes, and its yellowing leaf tips resemble other yellow-tinted images at this resolution.

- **bacterial_panicle_blight (F1 0.433)** is the smallest class (337 images). The disease shows on the panicle, a small region that 128 × 128 resizing and a blurry latent cannot resolve.
- **normal, hispa and blast**, the largest classes, are often confused with each other: 31 normal test images were predicted as hispa and 28 blast images as normal. Their differences are small lesions or scrape marks, which is exactly the detail the reconstructions lose.
- Small classes have higher recall than precision, the expected effect of the class-weighted loss.

7.5 Experiment Comparison

Table 9 compares every experiment on the validation split. For the autoencoder runs, "head val macro-F1" is a head trained the same way (learning rate 0.01, batch 64) on that run's frozen latents.

Table 9. Comparison of all experiments (validation split)

Exp.	Change (others at default)	Epochs	Best val MSE	Head val macro-F1	Observation
Start	η 0.01, batch 64, z 128	50	0.0441	0.2489	Still decreasing at epoch 50
E3a	η 0.1	50	0.0428	0.2328	Faster; stable
E3a	η 0.001	50	0.0573	–	Far too slow
E3b	η 0.1, batch 32	50	0.0386	0.3061	Best plain-SGD run at 50 epochs
E3b	η 0.1, batch 128	50	0.0433	–	Fewer updates per epoch
E2	η 0.1, z 64	50	0.0421	0.2073	
E2	η 0.1, z 256	50	0.0431	0.2992	Best latent size for classification
M	η 0.1, momentum 0.9	50	0.0254	0.4411	Much faster convergence
M	η 0.01, momentum 0.9	50	0.0428	–	
E1 (AE)	η 0.1, batch 32, z 128	150	0.0279	0.4646	Selected encoder
M	η 0.1, momentum 0.9	150	0.0223	0.4195	Lower MSE, worse head
H	Head η 0.001 on E1 encoder	50	–	0.2359	Too slow
H	Head η 0.1 on E1 encoder	19	–	0.5200	Selected head (final E1)

Two 50-epoch values (batch 32 and momentum 0.9 at η 0.1) are taken from the experiment log rows written before the 150-epoch runs reused those run names.

- **Learning rate** had the largest effect: η = 0.001 barely moved in 50 epochs, and η = 0.1 was fastest while staying stable.
- **Smaller batches** converged faster at a fixed learning rate, because batch 32 gives twice as many updates per epoch as batch 64.
- **Latent size:** at 50 epochs, a larger latent gave a better classifier (0.207, 0.233, 0.299 for 64, 128, 256), even though the reconstruction error was about the same.

- **Training length** mattered most: the selected plain-SGD encoder improved from 0.306 to 0.465 head macro-F1 between 50 and 150 epochs.
- **Momentum** reached a much lower MSE (0.0223) but a worse classifier after 150 epochs (0.419 vs 0.465), so lower reconstruction error did not mean better disease features.

7.6 Reconstruction Quality and Latent Space

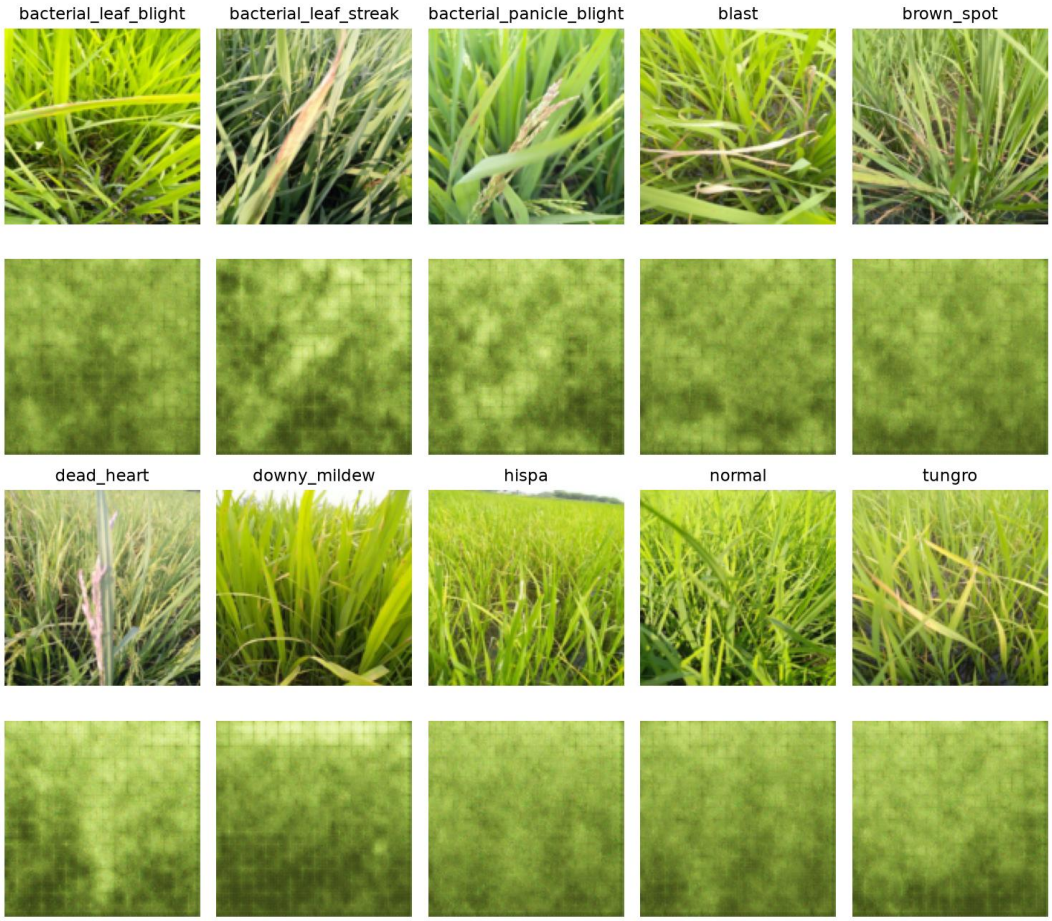


Figure 7. Test images (rows 1 and 3) and their reconstructions directly below (rows 2 and 4)

The reconstructions keep only the overall colour and the coarse light/dark layout. Individual leaves, leaf edges and lesions are **not** preserved, and a regular grid pattern is visible. The grid likely comes from the dense layer that expands z back to a 16 × 16 map, followed by stride-2 transposed convolutions, which are known to produce such artifacts [7]. **The PRD criterion "leaf shape and lesion regions visibly preserved" is not met.**

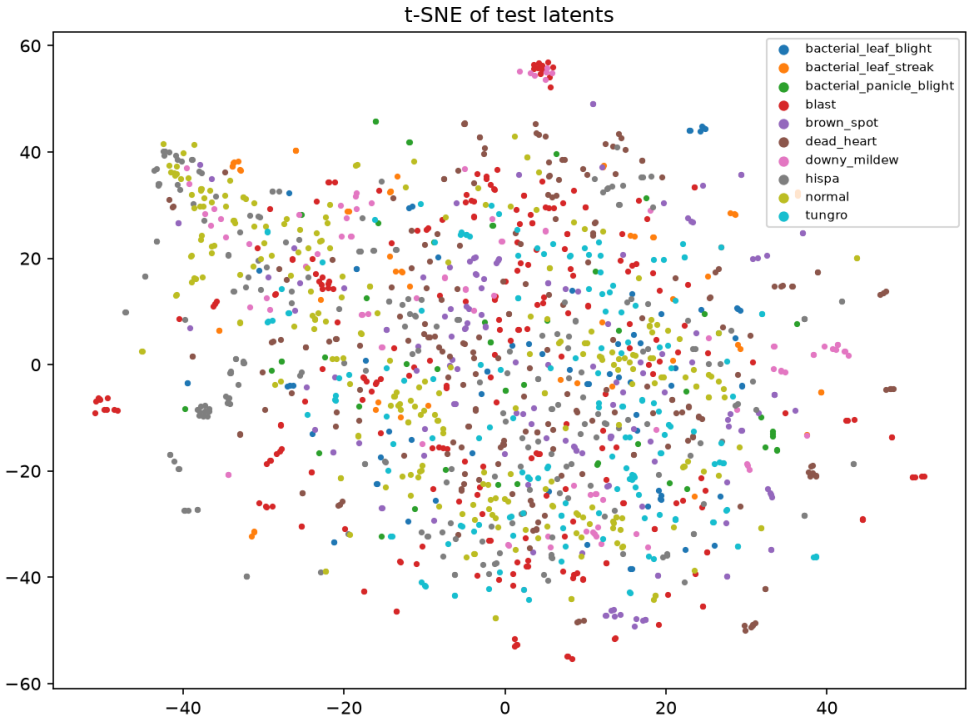


Figure 8. t-SNE of test latent vectors, coloured by class

The t-SNE projection [6] of the test latents is mostly one mixed cloud, so the classes are not cleanly separated. There are weak trends (normal, hispa and bacterial_leaf_streak toward the upper left) and a few small tight clusters, mainly of blast and hispa. These are probably photos from the same plot or session that look nearly identical without passing the duplicate threshold, and they may make some classes look easier than they are.

8. Discussion

8.1 Interpretation of Results

The learned representation helps, but modestly. E1 reaches 0.475 test macro-F1 against 0.370 for B1, and 0.847 AUC against 0.792, with the same 128-value representation. The AUC of 0.847 shows the classifier ranks the correct class fairly well even when its top choice is wrong.

The reconstructions and t-SNE explain why the gain is limited. The latent vector mostly encodes colour, brightness and coarse texture rather than lesion shapes, which is the PRD's listed risk “latent captures colour/background, not lesions”. Those global statistics are still more discriminative than linear PCA components, which is where the improvement over B1 comes from.

8.2 Effect of Architecture

Only the latent size was varied. At 50 epochs a larger latent gave a better classifier (0.207, 0.233 and 0.299 head macro-F1 for 64, 128 and 256), while the reconstruction error barely changed. A wider bottleneck keeps more information that the reconstruction loss does not reward but the classifier can use.

The bigger architectural limit is the bottleneck design. One dense layer squeezes a 32,768-value feature map into 128 numbers after only three convolutions, and it holds almost all of the 8.6 million parameters. Fine spatial detail is lost there, as the reconstructions show.

8.3 Effect of Optimization / Training

The learning rate was the most important setting, followed by the training length. Plain gradient descent sits on a plateau near the “average image” solution for about 25 epochs before it starts to learn structure, so short runs undersell it. Smaller batches helped at a fixed learning rate because they give more updates per epoch. Momentum 0.9 escaped the plateau much faster and reached a lower reconstruction error, but its latents classified slightly worse after 150 epochs. The main model therefore uses plain gradient descent, as assigned, and momentum is reported only as an ablation.

8.4 Overfitting / Underfitting

- **Autoencoder: underfitting rather than overfitting.** Training and validation MSE stay close (0.0254 vs 0.0279) and were still falling at epoch 150, and test MSE (0.0278) matches validation. More training or a larger model could still improve it.
- **Classifier head: overfits quickly.** Validation loss is lowest at epoch 11 while training accuracy keeps rising to 0.720. Validation accuracy kept improving slightly after that (peak 0.562 at epoch 18), so stopping on validation loss, as the PRD specified, may have stopped a little early for accuracy.
- **Validation vs test:** E1 scored 0.520 macro-F1 on validation and 0.475 on test. Many configurations were compared on the same validation split, so the validation number is optimistic. The test number, evaluated once, is the one to quote.

8.5 Objective Achievement

Table 10. Objective achievement

Objective	Status	Evidence
Prepare the data	Achieved	Dedup, grouped 70/15/15 split, saved indices (Section 3)
Build a CAE	Achieved, with a limitation	Trains and converges, but reconstructions lose lesions (Figure 7)
Train with mini-batch GD and document it	Achieved	Learning rate, batch size and convergence results (Tables 5, 9; Figure 5)
Classify from frozen latents vs a baseline	Achieved	E1 0.475 vs B1 0.370 test macro-F1 (Table 7)
Report loss, latent space, macro-F1, per-class	Achieved	Figures 5–8, Tables 7–8
Macro-F1 ≥ 0.60 (provisional target)	Not met	0.475 on test

9. Limitations

- **Dataset size and representativeness:** all images come from one dataset source, taken in similar field conditions. Performance on photos from other regions, cameras or seasons is unknown.
- **Class imbalance:** the classes range from 1,764 to 337 images. Class weighting improves recall on the small classes but costs precision (bacterial_leaf_blight precision 0.279).
- **Noise and near-duplicates:** 10 duplicate clusters had mixed labels, and the tight t-SNE clusters suggest some near-identical photos remain, possibly split across training and test.
- **Computational limits:** training ran on a laptop GPU, so images were reduced to 128 × 128 without keeping the aspect ratio. Lesions a few pixels wide in the original are mostly lost before the model sees them.
- **Limited hyperparameter search:** one factor at a time, 50-epoch sweeps while plain gradient descent was still improving, a single seed (42) with no repeats, and no fine-tuning run (E4).
- **Interpretability:** the 128 latent values have no direct meaning, and the blurry reconstructions show that the latent space does not represent lesions explicitly.
- **Generalisation:** the encoder was trained without labels, so nothing told it which details are diagnostic; MSE rewards average-looking reconstructions rather than disease features.

10. Conclusion

This project trained a convolutional autoencoder from scratch on 10,407 Paddy Doctor field images with plain mini-batch gradient descent, then classified ten disease classes from its frozen 128-value latent vectors. After a duplicate-aware 70/15/15 split and one-factor-at-a-time tuning, the selected model (learning rate 0.1, batch size 32, 150 epochs) reached a test reconstruction MSE of 0.0278 and a test macro-F1 of 0.475 (accuracy 0.484, AUC 0.847), beating a PCA plus logistic-regression baseline of the same size (0.370 macro-F1).

The unsupervised representation therefore carries useful disease signal, mostly colour and coarse texture. It does not preserve lesions, the reconstruction criterion was not met, and the provisional 0.60 macro-F1 target was not reached. Learning rate and training length were the decisive factors for plain mini-batch gradient descent.

10.1 Future Work

- Train a 150-epoch model with a 256-value latent, the best latent size for classification at 50 epochs.
- Run E4: unfreeze the encoder with a low learning rate to find the upper bound for this architecture.
- Add a fourth convolution layer or use a convolutional bottleneck, so less spatial detail is discarded.
- Use larger inputs (for example 224 × 224) or crop patches around leaves.
- Repeat the key runs over several seeds and report the mean and standard deviation.
- Test on photos from other regions and devices before any field use.

11. References

[1] Petchiammal A., Briskline Kiruba S., Murugan D., & Pandarasamy A. (2023). Paddy Doctor: A visual image dataset for automated paddy disease classification and benchmarking. In Proceedings of the 6th Joint International Conference on Data Science & Management of Data (CODS-COMAD 2023). ACM. DOI: 10.1145/3570991.3570994

[2] Kaggle. (2022). Paddy Doctor: Paddy Disease Classification [Competition data]. https://www.kaggle.com/competitions/paddy-disease-classification

[3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536. DOI: 10.1038/323533a0

[4] Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. Science, 313(5786), 504–507. DOI: 10.1126/science.1127647

[5] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 1026–1034.

[6] van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. Journal of Machine Learning Research, 9, 2579–2605.

[7] Odena, A., Dumoulin, V., & Olah, C. (2016). Deconvolution and checkerboard artifacts. Distill. DOI: 10.23915/distill.00003

[8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[9] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.

[10] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

APPENDIX A – SOURCE CODE / IMPORTANT CODE SNIPPETS

GitHub repository: [GITHUB REPOSITORY LINK]

APPENDIX B – ADDITIONAL RESULTS

Table 11 lists every run in runs/experiment_log.csv (the latest row for each run name). Times per epoch are higher for runs that shared the GPU with a second run.

Table 11. Experiment log

Run	Stage	Epochs	Best val loss	Val macro-F1	s / epoch
z128_b64_lr0.01_m0.0	ae	50	0.0441	–	5.4
z128_b64_lr0.01_m0.0 → head_b64_lr0.01	head	42	2.1019	0.249	0.5
z128_b64_lr0.1_m0.0	ae	50	0.0428	–	5.4
z128_b64_lr0.001_m0.0	ae	50	0.0573	–	5.1
z128_b64_lr0.1_m0.9	ae	150	0.0223	–	13.3
z128_b32_lr0.1_m0.0	ae	150	0.0279	–	12.2
z128_b128_lr0.1_m0.0	ae	50	0.0433	–	8.7
z128_b64_lr0.01_m0.9	ae	50	0.0428	–	11.5
z128_b64_lr0.1_m0.9 → head_b64_lr0.01	head	50	1.6486	0.419	0.3
z128_b32_lr0.1_m0.0 → head_b64_lr0.01	head	50	1.5325	0.465	0.3
z128_b64_lr0.1_m0.0 → head_b64_lr0.01	head	40	2.0731	0.233	0.4
z64_b64_lr0.1_m0.0	ae	50	0.0421	–	9.7
z256_b64_lr0.1_m0.0	ae	50	0.0431	–	18.6
z64_b64_lr0.1_m0.0 → head_b64_lr0.01	head	50	2.1052	0.207	0.5
z256_b64_lr0.1_m0.0 → head_b64_lr0.01	head	33	2.0005	0.299	0.5
z128_b32_lr0.1_m0.0 → head_b64_lr0.1	head	19	1.4815	0.520	0.4
z128_b32_lr0.1_m0.0 → head_b64_lr0.001	head	50	2.0432	0.236	0.3